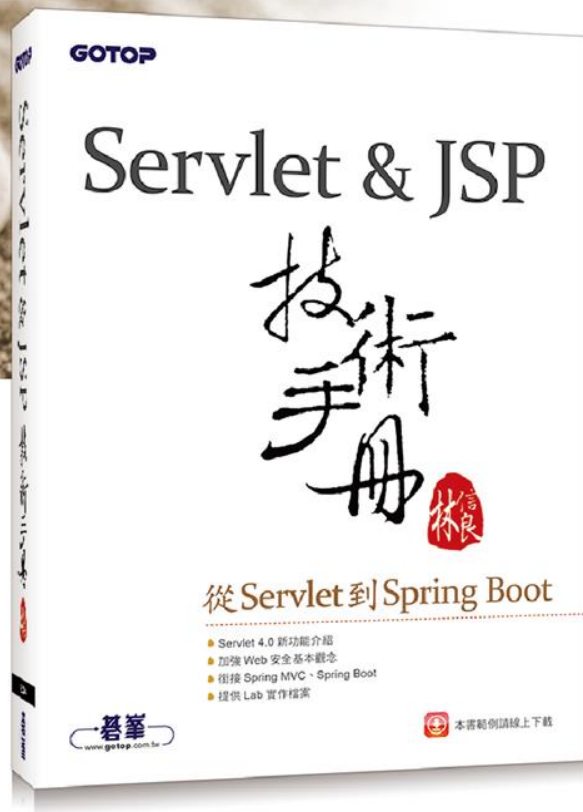




Servlet & JSP

技術手冊

從Servlet到Spring Boot

- Servlet 4.0 新功能介紹
- 加強 Web 安全基本觀念
- 銜接 Spring MVC、Spring Boot
- 提供 Lab 實作檔案

CHAPTER 12

• Spring起步走



學習目標

- 使用Gradle
- 結合Gradle與IDE
- 認識相依注入
- 使用Spring核心

使用Gradle

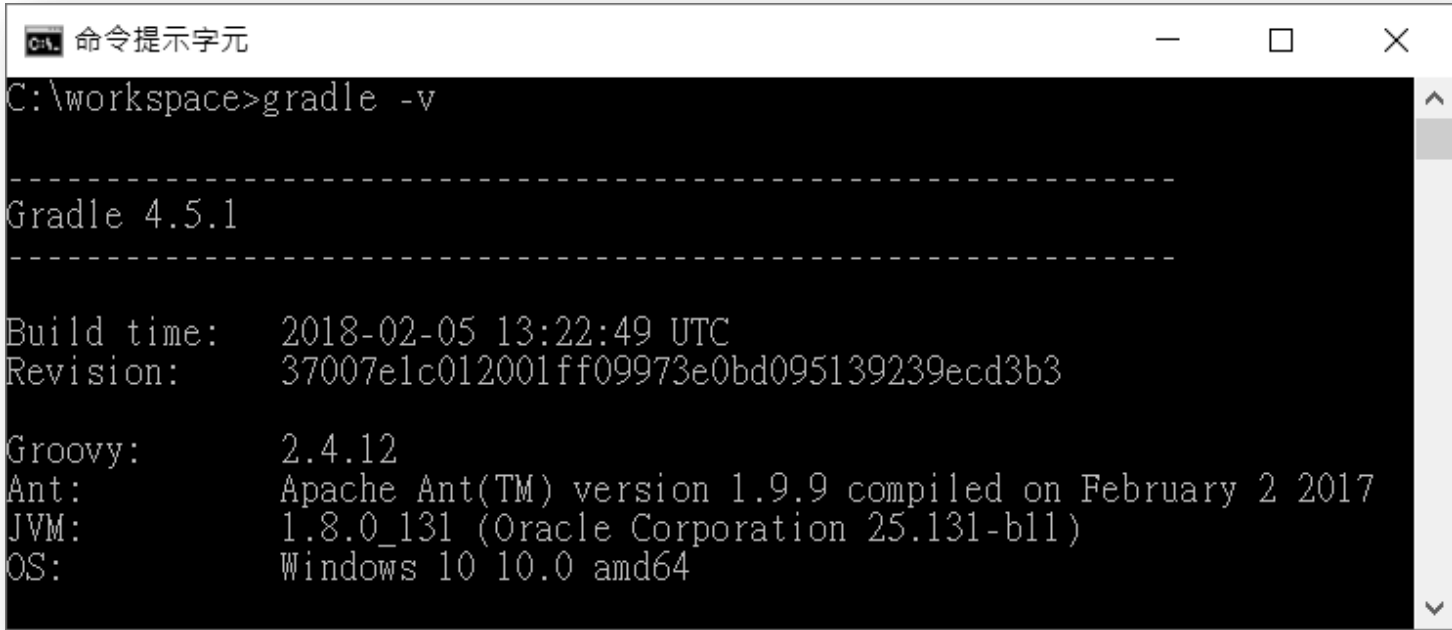
- 在Spring 3.x或之前的版本中，可以在Spring官方網站 直接下載JAR檔案
- 從4.x開始，推薦使用Gradle或Maven來下載，Spring本身是使用Gradle來管理

下載、設定Gradle

- Ant (Another Neat Tool)
- Maven
- Ant 結合 Ivy
- Gradle

下載、設定Gradle

- Gradle | Release
 - gradle.org/releases/



```
命令提示字元
C:\workspace>gradle -v

-----
Gradle 4.5.1
-----

Build time:   2018-02-05 13:22:49 UTC
Revision:     37007e1c012001ff09973e0bd095139239ecd3b3

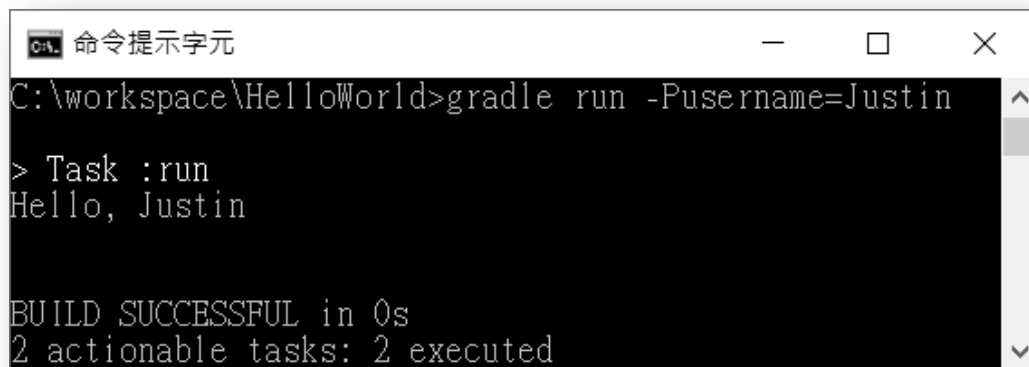
Groovy:       2.4.12
Ant:          Apache Ant(TM) version 1.9.9 compiled on February 2 2017
JVM:          1.8.0_131 (Oracle Corporation 25.131-b11)
OS:           Windows 10 10.0 amd64
```

簡單的Gradle專案

- Gradle的慣例期待.java原始碼會置放在src\main\java資料夾，依套件階層放置

```
apply plugin: 'java'
apply plugin: 'application'
mainClassName = "cc.openhome.Main"

run {
    args username
}
```



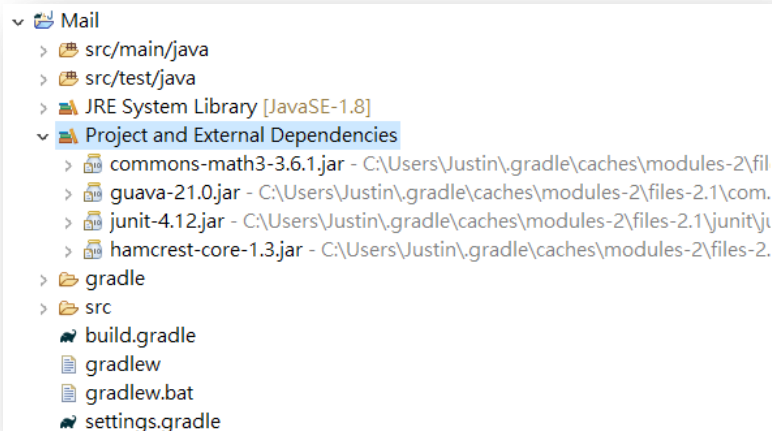
A screenshot of a Windows command prompt window titled "命令提示字元". The window shows the following text:

```
C:\workspace\HelloWorld>gradle run -Pusername=Justin
> Task :run
Hello, Justin

BUILD SUCCESSFUL in 0s
2 actionable tasks: 2 executed
```

Gradle與Eclipse

- 使用Eclipse內建的Gradle Project



```
apply plugin: 'java-library'
```

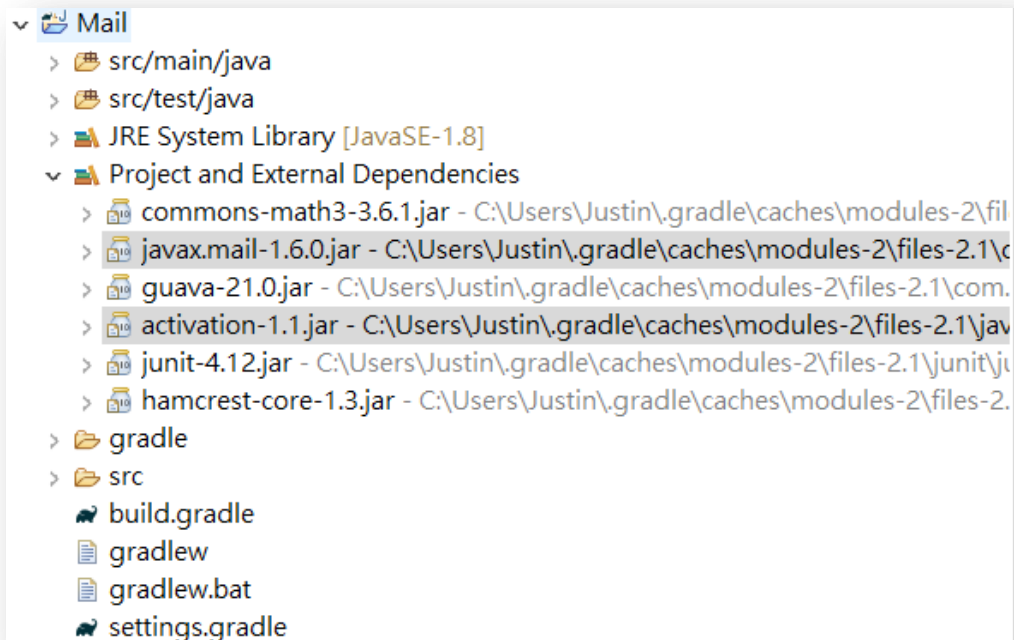
```
repositories {  
    // 預設的相依程式庫來源  
    jcenter()  
}
```

```
dependencies {  
    // 這邊可以宣告相依的程式庫資訊  
    api 'org.apache.commons:commons-math3:3.6.1'  
    implementation 'com.google.guava:guava:21.0'  
    testImplementation 'junit:junit:4.12'  
}
```

```
apply plugin: 'java-library'
```

```
repositories {  
    jcenter()  
}
```

```
dependencies {  
    // https://mvnrepository.com/artifact/com.sun.mail/javax.mail  
    compile group: 'com.sun.mail', name: 'javax.mail', version: '1.6.0'  
  
    api 'org.apache.commons:commons-math3:3.6.1'  
    implementation 'com.google.guava:guava:21.0'  
    testImplementation 'junit:junit:4.12'  
}
```



Gradle與Eclipse

- 既有的Java應用程式專案，可以直接在專案上按右鍵執行「Configure/Add Gradle Nature」
- 在專案上按右鍵執行「New/File」建立build.gradle檔案

認識Spring核心

- 整個Spring框架是非常龐大的，試圖完全掌握沒有意義
- 從Spring的核心開始認識，初步運用Spring來解決一些問題

相依注入

```
public class Main {
    public static void main(String[] args) {
        // JdbcDataSource 實現了 DataSource 介面
        JdbcDataSource dataSource = new JdbcDataSource();
        dataSource.setURL(
            "jdbc:h2:tcp://localhost/c:/workspace/SpringDI/gossip");
        dataSource.setUser("caterpillar");
        dataSource.setPassword("12345678");

        AccountDAO acctDAO = new AccountDAOJdbcImpl(dataSource);
        MessageDAO messageDAO = new MessageDAOJdbcImpl(dataSource);

        UserService userService = new UserService(acctDAO, messageDAO);

        userService.messages("caterpillar")
            .forEach(message -> {
                System.out.printf("%s\t%s%n",
                    message.getLocalDateTime(),
                    message.getBlabla());
            });
    }
}
```

相依注入

- 物件的建立與相依注入（**Dependency Injection**）是必要的關切點
- 當過程太過冗長，模糊了商務流程之時，應該適當地將之分離

相依注入

```
public class Service {  
    public static UserService getUserService() {  
        JdbcDataSource dataSource = new JdbcDataSource();  
        dataSource.setURL(  
            "jdbc:h2:tcp://localhost/c:/workspace/SpringDI/gossip");  
        dataSource.setUser("caterpillar");  
        dataSource.setPassword("12345678");  
  
        AccountDAO acctDAO = new AccountDAOJdbcImpl(dataSource);  
        MessageDAO messageDAO = new MessageDAOJdbcImpl(dataSource);  
  
        UserService userService = new UserService(acctDAO, messageDAO);  
        return userService;  
    }  
}
```

相依注入

```
public class Main {  
    public static void main(String[] args) {  
        UserService userService = Service.getUserService();  
  
        userService.messages("caterpillar")  
            .forEach(message -> {  
                System.out.printf("%s\t%s%n",  
                    message.getLocalDateTime(),  
                    message.getBlabla());  
            });  
    }  
}
```

相依注入

- 隨著打算開始整合各種程式庫或方案，你會遇到各種物件建立與相依設定需求
- 為此，你可能會重構Service，使之越來越通用，像是可透過組態檔來進行相依設定
- 甚至成為一個通用於各式物件建立與相依設定的容器
- 最有名的實現之一就是Spring框架

使用Spring核心

```
apply plugin: 'java-library'

repositories {
    jcenter()
}

dependencies {
    testImplementation 'junit:junit:4.12'

    // https://mvnrepository.com/artifact/com.h2database/h2
    compile('com.h2database:h2:1.4.196')

    // https://mvnrepository.com/artifact/org.springframework/spring-context
    compile('org.springframework:spring-context:5.0.3.RELEASE')
}
```


使用Spring核心

```
@Configuration
@ComponentScan
public class AppConfig {
    @Bean
    public DataSource getDataSource() {
        JdbcDataSource dataSource = new JdbcDataSource();
        dataSource.setURL(
            "jdbc:h2:tcp://localhost/c:/workspace/gossip/gossip");
        dataSource.setUser("caterpillar");
        dataSource.setPassword("12345678");
        return dataSource;
    }
}
```

使用Spring核心

@Component

```
public class AccountDAOJdbcImpl implements AccountDAO {  
    @Autowired  
    private DataSource dataSource;  
    ...略  
}
```

@Component

```
public class MessageDAOJdbcImpl implements MessageDAO {  
    @Autowired  
    private DataSource dataSource;  
    ...略  
}
```

@Component

```
public class UserService {  
    @Autowired  
    private final AccountDAO acctDAO;  
  
    @Autowired  
    private final MessageDAO messageDAO;  
  
    ...略  
}
```

使用Spring核心

```
public class Main {  
    public static void main(String[] args) {  
        ApplicationContext context =  
            new AnnotationConfigApplicationContext(  
                cc.openhome.AppConfig.class);  
  
        UserService userService = context.getBean(  
            cc.openhome.model.UserService.class);  
  
        userService.messages("caterpillar")  
            .forEach(message -> {  
                System.out.printf("%s\t%s%n",  
                    message.getLocalDateTime(),  
                    message.getBlabla());  
            });  
    }  
}
```